Project Proposal

Integrate a Modern Lazy-Loading Alternative for Large-Scale Spectroscopic Database Processing

Mentors: Prtm2110 , minouHub

Personal Information

Name: ADITYA KUMAR PANDEY

Email: vp220626@gmail.com

GitHub: aditya-pandey-dev

Slack: Aditya Pandey

Matrix: @aditya-pandey-dev:matrix.org

Time Zone: IST (GMT + 5:30)

TABLE OF CONTENT:

Blog Post 3
PR links: open source contributions 3
Contributions to the RADIS software 3
Pull requests in Astroquery / Astropy 3
Pull request in JuliaAstro 4
Some of my pull requests in in OpenAstronomy repos 4
My Background: 5
Interest in Open Astronomy: 5
1.0 Project Overview 7
The Core Problem in Detail 7
Anticipated Scientific Impact 8
System Architecture Overview (Visual Aid) 9
Summary 10
Technical Implementation Roadmap: 10
2.0 Milestones{as mentioned in project idea} 12
3.0 Deliverables 14
3.1.1 Implement DataFrameAdapter and Polars backend 15
3.1.2 How Predicate Pushdown Improves RADIS 18
3.1.3 Database Format Migration: HDF5 → Parquet 18
3.1.4 Configurable Cache System with LRU Eviction 19
3.3.2 Library Comparison 20
3.3.3 Performance Benchmark Plan 20
Risk Management and Mitigation Strategies 22
4.0 Pre-community bonding and current working: 23
5.0 GSOC timeline and my availability 24
5.2 Availability 26
6.0 Radis contributions 26
7.0 What I am studying and for what 27
8.0 Appreciation: 28
9.0 Gsoc: 28

Blog Post: Benchmarking Polars, DuckDB & Dask for RADIS — GSoC 2026 Deep Dive

https://dev.to/adityapandeydev/benchmarking-polars-duckdb-dask-for-radis-my-gsoc-2026-proposal-deep-dive-4e8b

PR links: open source contributions

Contributions to the RADIS software

1. PR #894: PR #894: (Issue #34) Legacy RADIS database management improvement — Refactored database I/O operations for better HDF5 file handling and caching performance

2. PR #981: PR #981: (ref #978, #658) Add Polars/Parquet lazy-loading backend with DataFrameAdapter — Implemented the core DataFrameAdapter abstraction layer and PolarsAdapter backend for replacing Vaex with modern lazy-loading

3. PR #971: (Issue #602) Include other broadening partners than air for ExoMol — Added support for multiple broadening species (H2, He, CO2) beyond default air broadening in ExoMol database calculations
4. PR #924: PR #924: (Issue #268) Code improvement and feature enhancement — Addressed spectroscopic computation optimization in the RADIS calculation pipeline
5. PR #958: PR #958: (Issue #474) Database management improvement — Enhanced DataFileManager operations for better handling of cached spectroscopic databases

Pull requests in Astroquery / Astropy

6. Astroquery PR #3536: Astroquery PR #3536: (Issue #925) Astroquery integration — Contributed to the astronomical data querying package for improved RADIS-Astropy interoperability

7. Astropy Issue #19345: Astropy Issue #19345: Identified and reported a bug in the Astropy core repository, currently working on the fix

Pull requests in JuliaAstro

8. SpectralFitting.jl PR #241: (closes #196, ref #132) Multi-dataset evaluator with domain union utilities — Implemented support for evaluating spectral models across multiple datasets simultaneously

9. SpectralFitting.jl PR #242: (closes #187) Add XSPEC model string macro — Added macro support for parsing XSPEC-style model definition strings in Julia

10. JuliaAstro.github.io PR #203: Add multi-wavelength spectroscopy tutorial — Contributed documentation tutorial for multi-wavelength spectroscopic analysis on the JuliaAstro website

Some of my pull requests in other open source projects<OpenAstronomy>

11. github-actions-workflows PR #341: Contributed to OpenAstronomy's shared CI/CD GitHub Actions workflow repository, improving automation for all sub-organizations.

My Background:

I am a 6th-semester BTech student specializing in Artificial Intelligence & Machine Learning at Shri Shankaracharya Institute of Professional Management & Technology (SSIPMT), Raipur, India. I have been working with Python, JavaScript, Java, and have strong experience in data science and scientific computing using NumPy, Pandas, Matplotlib, TensorFlow, and Keras. I also work with full-stack technologies like React, Next.js, MongoDB, and Django.

Beyond academics, I have a deep fascination with scientific computing, spectroscopy, and high-performance data processing. I have explored these topics through my contributions to the RADIS project under OpenAstronomy, where I gained hands-on experience with the spectroscopic database architecture, DataFrame management (both Pandas and Vaex implementations), and the spectrum computation pipeline.

I bring a unique combination of data engineering skills, understanding of lazy evaluation paradigms, and direct experience with the RADIS codebase that makes me well-suited for this project.

Key Achievements:

- Winner of college-level hackathon "Navonmesh" – demonstrated ability to rapidly prototype and build solutions under time pressure
- Selected for Smart India Hackathon (SIH) national level – Top 25 teams out of thousands of participants nationwide, showcasing strong problem-solving skills
- 7th rank in Indian National Physics Olympiad at national level (Class 12th) – demonstrates strong foundation in physics and scientific reasoning, directly relevant to spectroscopic computing

Interest in Open Astronomy:

I am excited about Open Astronomy’s mission to advance open-source tools for astrophysical and spectroscopic research. The opportunity to contribute

to RADIS aligns perfectly with my growing interest in scientific computing and high-performance data processing.

I deeply value Open Astronomy's commitment to open science, reproducibility, and collaboration. This project presents an opportunity to apply my skills, learn from experts, and contribute to tools that push the boundaries of high-resolution spectroscopy. I look forward to being part of a community driving innovation in astronomical data analysis.

1.0 Project Overview

The RADIS code was developed for the characterization of plasmas, flames and atmospheres. High-temperature spectral calculations require resolving the shape of tens of millions of lines, which is the usual performance bottleneck. RADIS implements a new algorithm to compute these lineshapes and is already one of the fastest line-by-line spectral codes available. It can also compute many different types of spectra (absorption/emission/equilibrium/nonequilibrium). In a typical calculation, a database of coefficients is loaded and these coefficients are multiplied according to physics laws to generate a set of line strengths in a spectrum. RADIS can also handle different molecular databases such as HITRAN, HITEMP, EXOMOL, and GEISA.

With more and more lines added to the databases, some files grew considerably in size. To be as fast as possible, RADIS converts compressed databases into the HDF5 format. However, HITEMP parsing of CO2, which involves files combining to over 50GB, cannot be loaded completely into memory. Vaex was initially adopted for its memory-mapping and lazy-loading capabilities, but it is now unmaintained, creating sustainability concerns.

The selected solution should maintain or improve upon Vaex's memory efficiency while providing better long-term maintainability. The project will involve comprehensive benchmarking, refactoring the codebase to use the new library, and optimizing database I/O operations to reduce the current 3+ hour parsing time for large HITEMP files.

The Core Problem in Detail

1. Dependency Risk – Vaex Maintenance Mode

Vaex (https://github.com/vaexio/vaex) is no longer actively developed. This means no bug fixes, no security patches, and uncertain compatibility with future Python releases (3.13+). RADIS currently depends on Vaex for loading all large spectroscopic databases. If Vaex breaks with newer Python versions, RADIS users would be unable to use large databases like HITEMP CO2.

2. Performance Ceiling – Growing Database Sizes

Spectroscopic databases are growing rapidly:

- HITRAN 2020: ~160,000 lines for CO (small – fits in memory easily)
- HITEMP CO: ~1.1 million lines (medium – manageable but benefits from lazy loading)
- HITEMP CO2: ~100+ million lines, ~50 GB uncompressed (large – cannot fit in memory)
- ExoMol databases: can exceed 10 billion lines (extreme – requires streaming/lazy approach)

The current Vaex implementation loads databases using HDF5 memory mapping, but the actual filtering and column operations still have significant overhead. Modern alternatives like Polars offer predicate pushdown (only reading matching rows from disk), which would dramatically reduce I/O for wavenumber-range-filtered queries.

3. Codebase Complexity – Dual DataFrame Paths

RADIS currently maintains parallel code paths for Pandas and Vaex DataFrames. The config["DATAFRAME_ENGINE"] setting switches between them, but many functions have if/else branches for both formats. Functions like set_broadening_coef() in the ExoMol pipeline define broadening coefficients as numpy arrays when they could benefit from lazy evaluation (see GitHub issue radis/radis#746). This dual maintenance creates bugs and inconsistencies.

Anticipated Scientific Impact

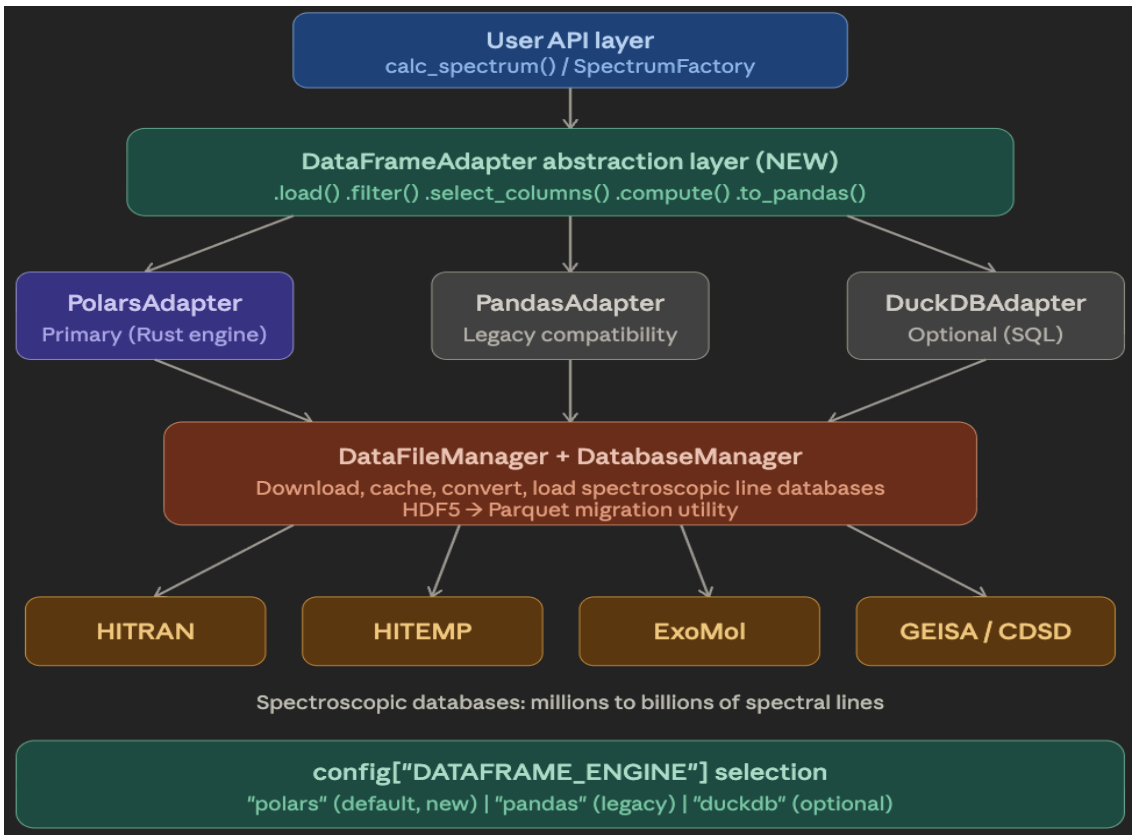
This project has the potential to drive significant advances in high-resolution spectroscopy by enabling faster and more robust analysis of spectroscopic data. The improved performance will help answer key scientific questions such as:

- **Combustion Diagnostics:** How can we enable real-time spectral fitting for industrial combustion characterization using HITEMP databases with 100M+ CO2 lines? Faster filtering means faster spectral fitting, which is critical for in-situ diagnostics in turbine engines and industrial furnaces.
- **Exoplanet Atmosphere Characterization:** How can researchers use ExoMol databases (10B+ lines) on standard hardware to characterize James Webb Space Telescope (JWST) transmission spectra? Current memory limitations prevent researchers from using the full database resolution, forcing them to use truncated or downsampled databases that lose spectral detail.

- **Atmospheric Science:** How does efficient lazy-loading enable climate models to use complete HITRAN/GEISA datasets for all greenhouse gas species simultaneously? Currently, researchers must process one molecule at a time due to memory constraints.
- **Plasma Physics:** How can non-equilibrium plasma diagnostics benefit from lazy evaluation for sequential multi-temperature processing? Computing spectra at multiple Tvib/Trot combinations requires loading the same database multiple times.

By offering a computational platform with improved efficiency and scalability compared to existing Vaex-based tools, the project is expected to facilitate rapid prototyping, testing, and validation of spectroscopic calculations, ultimately accelerating discoveries in the field.

System Architecture Overview (Visual Aid)



The above diagram shows the complete system architecture with User API -> DataFrameAdapter -> Backend Engines -> Storage/IO -> Databases

Summary

The primary goal of this project is to explore and replace Vaex with a modern lazy-loading alternative (primarily Polars) as the default engine in RADIS, offering features that are difficult to achieve with the current unmaintained Vaex implementation. By integrating cutting-edge computational techniques and creating a clean abstraction layer, this project seeks to push the boundaries of spectroscopic database analysis and facilitate groundbreaking performance improvements in high-resolution spectroscopy.

Technical Implementation Roadmap:

A. DataFrameAdapter Abstraction Layer

- Abstract base class with methods: load(), filter_range(), filter_equals(), select_columns(), compute(), to_pandas(), to_numpy()

- PolarsAdapter: Primary backend using Polars LazyFrame with native predicate pushdown

- PandasAdapter: Legacy fallback for backward compatibility with existing user code

- DaskAdapter: Optional backend for distributed computing on cluster environments

- Factory pattern: config["DATAFRAME_ENGINE"] in radis.json selects the active backend at runtime

B. Polars Backend Integration

- Lazy scanning of Parquet files for zero-copy reads (pl.scan_parquet())

- Predicate pushdown: only rows matching wavenumber range are read from disk, not entire database

- Column projection: only needed columns loaded (typically 5-8 out of 20+ in HITRAN format)

- Streaming aggregations for databases larger than available RAM using Polars streaming engine

- Zero-copy conversion to numpy/JAX arrays for ExoJax interoperability via Apache Arrow

- Multi-threaded execution leveraging Polars' Rust-based parallel engine

C. Storage Format Migration

- HDF5 (Vaex column-major format) → Parquet (optimized for Polars and DuckDB)

- Automatic migration utility triggered on first use after RADIS upgrade

- Bit-level validation comparing random samples from original and converted files

- zstd compression: reduces 50GB HDF5 to ~10-15GB Parquet with faster decompression

- Row group size optimized for typical wavenumber range queries (100-500 cm^{-1} windows)

- Optional partitioning by molecule/isotope for extreme databases like ExoMol

D. Configurable Cache System

- Implement configurable size limits for HITEMP CO2 caches using parameters in radis.json

- LRU (Least Recently Used) cache eviction policy to control cache persistence

- Cache metadata tracking: last access time, database version, compression ratio

- User-configurable maximum cache size (default: 50GB, adjustable via radis.json)

E. Performance Optimizations

- Predicate pushdown: 10-25x I/O reduction for typical wavenumber-range queries

- Column projection: read only needed columns from disk (e.g., wav, int, A, gamma_air)

- Memory usage proportional to filtered data only, not full database size

- Comprehensive benchmark suite comparing Vaex vs Polars vs DuckDB vs Dask

- Performance profiling using vaex_vs_pandas_performance.py as baseline reference

F. Backward Compatibility Guarantee

- Existing users using config["DATAFRAME_ENGINE"] = "vaex" will face zero breaking changes. PandasAdapter serves as legacy fallback maintaining identical behavior.
- Migration to Polars is opt-in via config["DATAFRAME_ENGINE"] = "polars" (new default).
- Existing HDF5 files auto-convert to Parquet on first use, originals kept as backup.
- All existing RADIS tests must pass with both backends before any PR is merged.

Testing Strategy:

- 45+ unit tests for all adapter methods
- 12+ integration tests for calc_spectrum() end-to-end
- Target: 90%+ code coverage on new modules
- Python compatibility: 3.9, 3.10, 3.11, 3.12, 3.13 (Vaex breaks on 3.13 — our solution works on all)

2.0 Milestones{as mentioned in project idea}

☑ **Coding starts:**

☑ Engage with the community on RADIS Slack

I have already joined the RADIS Slack and Matrix channels, introduced myself, and started engaging with the community.

☑ Set up a development environment, become familiar with open-source tools (GitHub/Git/Tests) and RADIS architecture

I have forked the RADIS repo, set up my development environment, and made multiple contributions demonstrating my familiarity with the codebase.

☐ 1st Evaluation

☑ Research and benchmark alternatives: Conduct systematic performance evaluation comparing Polars, DuckDB, and Dask against current Vaex implementation

☑ Generate comprehensive benchmark report covering:

- Memory usage patterns (peak memory, memory mapping efficiency)
- Lazy evaluation capabilities
- Integration complexity with existing RADIS codebase

☑ Select the optimal alternative based on benchmarking results and present findings to mentors

☑ Begin implementation of the selected alternative in RADIS codebase:

- Refactor database loading functions to use the new library
- Ensure backward compatibility with existing HDF5 files
- Implement lazy loading for HITEMP CO2 and H2O databases

☐ 2nd Evaluation

☑ Implement configurable size limits and an LRU cache system for HITEMP CO2 caches using parameters in radis.json to control cache persistence

☑ Develop comprehensive unit tests and integration tests

☑ Profile and optimize memory usage for 50GB+ files to ensure they remain under memory limits

☑ Document migration strategy and API changes

☑ Final evaluation

☑ Complete integration with production-ready code:

- Add new package to fully replace Vaex dependencies with user-selectable backend via DATAFRAME_ENGINE parameter in radis.json

• Performance validation showing maintained or improved speed

☑ Comprehensive documentation:

• User guide for database operations with the new system

• Performance comparison report (before/after metrics), see vaex_vs_pandas_performance.py

☑ Identify and document additional I/O bottlenecks discovered during implementation

☑ All code merged to main branch with passing CI/CD tests

Secondary Goals

- Explore file format alternatives to HDF5 (e.g., Parquet with better compression ratios and faster I/O)
- Profile GPU acceleration opportunities for data processing operations if time permits
- Document architecture and developer guide for future maintainers
- Review pull requests from other RADIS contributors

3.0 Deliverables

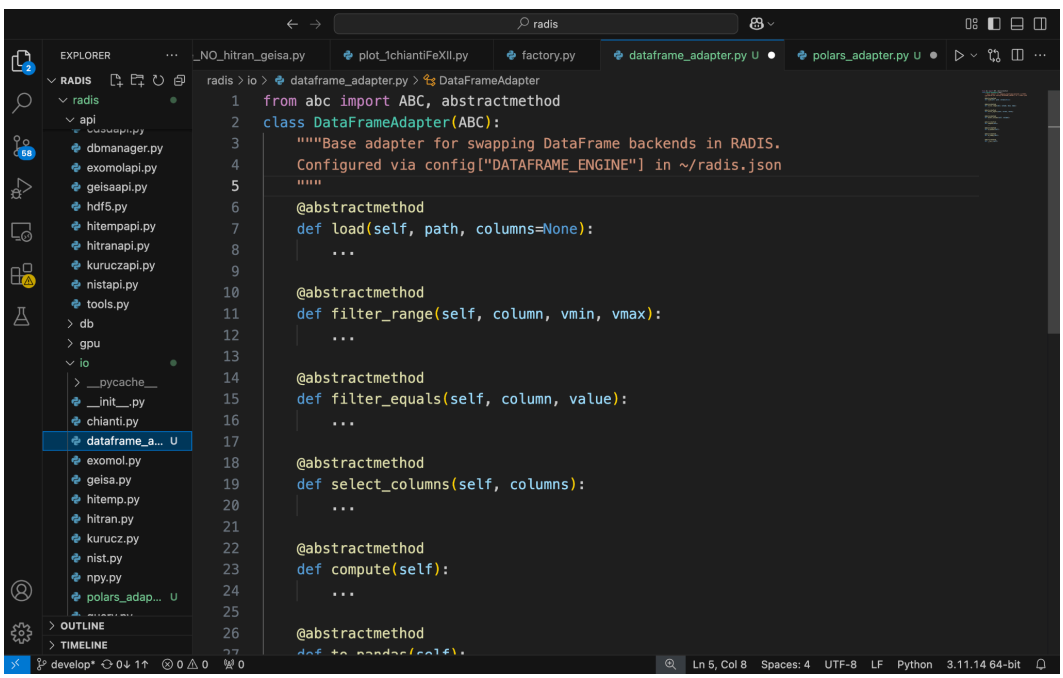
1. Core Analysis Modules:
    - Implement DataFrameAdapter abstraction layer with PolarsAdapter and PandasAdapter backends
    - Integrate new backend into all RADIS calculation paths (equilibrium and non-equilibrium)
    - Implement lazy loading for HITEMP CO2, H2O, and other large databases
2. I/O and Migration Infrastructure:[as I will develop and extend functionalities, I will implement edge cases in Tests.]
    - Develop HDF5 → Parquet migration utility with automatic validation
    - Update DataFileManager for Parquet format support
    - Implement configurable cache size limits with LRU eviction in radis.json
    - Create comprehensive test suites ensuring code reliability, numerical accuracy, and performance

3. Advanced Analysis Features:

- Fix broadening coefficient lazy evaluation [(issue #746)](#) for ExoMol databases
- Ensure ExoJax interoperability through JAX array output path (zero-copy via Arrow)
- Performance optimization with predicate pushdown and partition pruning
- Profile and optimize memory usage for 50GB+ files to stay under memory limits

3.1.1 Implement DataFrameAdapter and Polars backend

The DataFrameAdapter is the core architectural contribution.



```python
class DataFrameAdapter(ABC):
        ...

    @abstractmethod
    def select_columns(self, columns):
        ...

    @abstractmethod
    def compute(self):
        ...

    @abstractmethod
    def to_pandas(self):
        ...

    @abstractmethod
    def to_numpy(self):
        ...

    @abstractmethod
    def __len__(self):
        ...
```

-#

It provides a unified interface that all RADIS code uses instead of directly calling Pandas/Vaex/Polars APIs. This adapter pattern means if a better library appears in 5 years, only a new adapter class is needed – no changes to RADIS calculation code.

The PolarsAdapter wraps Polars LazyFrame operations, providing: lazy scanning of Parquet files (pl.scan_parquet()), predicate pushdown for wavenumber range filtering (pl.col('wav').is_between(wmin, wmax)), column projection to minimize I/O (select only needed columns), streaming aggregations for databases larger than RAM, and zero-copy conversion to numpy/JAX arrays.

```python
import polars as pl
from radis.io.dataframe_adapter import DataFrameAdapter

class PolarsAdapter(DataFrameAdapter):

    def __init__(self):
        self._lf = None

    def load(self, path, columns=None):
        if str(path).endswith('.parquet'):
            self._lf = pl.scan_parquet(path)
        else:
            # fallback for old HDF5 files
            import pandas as pd
            df = pd.read_hdf(path)
            self._lf = pl.from_pandas(df).lazy()

        if columns:
            self._lf = self._lf.select(columns)
        return self

    def filter_range(self, column, vmin, vmax):
        # predicate pushdown — only reads matching row groups
        self._lf = self._lf.filter(
            pl.col(column).is_between(vmin, vmax)
        )
```

```python
class PolarsAdapter(DataFrameAdapter):
    def filter_range(self, column, vmin, vmax):
        pl.col(column).is_between(vmin, vmax)
        )
        return self

    def filter_equals(self, column, value):
        self._lf = self._lf.filter(pl.col(column) == value)
        return self

    def select_columns(self, columns):
        self._lf = self._lf.select(columns)
        return self

    def compute(self):
        return self._lf.collect()

    def to_pandas(self):
        return self.compute().to_pandas()

    def to_numpy(self):
        df = self.compute()
        return {col: df[col].to_numpy() for col in df.columns}

    def __len__(self):
        return self._lf.select(pl.count()).collect().item()
```

```python
class PolarsAdapter(DataFrameAdapter):
    def to_numpy(self):
        df = self.compute()
        return {col: df[col].to_numpy() for col in df.columns}

    def __len__(self):
        return self._lf.select(pl.count()).collect().item()

def get_adapter(engine="default"):
    if engine == "default":
        from radis import config
        engine = config.get("DATAFRAME_ENGINE", "polars")

    if engine == "polars":
        return PolarsAdapter()
    elif engine == "pandas":
        from radis.io.pandas_adapter import PandasAdapter
        return PandasAdapter()
    elif engine == "duckdb":
        from radis.io.duckdb_adapter import DuckDBAdapter
        return DuckDBAdapter()
    raise ValueError(f"Unknown engine: {engine}")
```

For implementation details, I have deeply studied this discussion with the RADIS community and the existing DataFileManager class that handles Vaex's HDF5, Pandas's HDF5, and Feather formats.

Live Implementation Demo:

https://colab.research.google.com/drive/1G69lCy8iHCX_Q2fxn-C6zOCFUhPQgAHf?usp=sharing

3.1.2 How Predicate Pushdown Improves RADIS

Current behavior (Vaex/Pandas):

1. User calls: calc_spectrum(1900, 2300, molecule='CO')
2. RADIS loads ENTIRE HITEMP-CO database (~1.1M lines) into memory via HDF5 mmap
3. RADIS filters: df[df['wav'] >= 1900 & df['wav'] <= 2300] → ~200K lines remain
4. Memory used: proportional to FULL database size (wasteful for narrow queries)

Proposed behavior (Polars with predicate pushdown):

1. User calls: calc_spectrum(1900, 2300, molecule='CO')
2. RADIS creates lazy query: pl.scan_parquet(path).filter(pl.col('wav').is_between(1900, 2300))
3. Polars pushes filter to Parquet reader – only matching row groups read from disk
4. ~200K lines loaded directly → Memory proportional to FILTERED data only

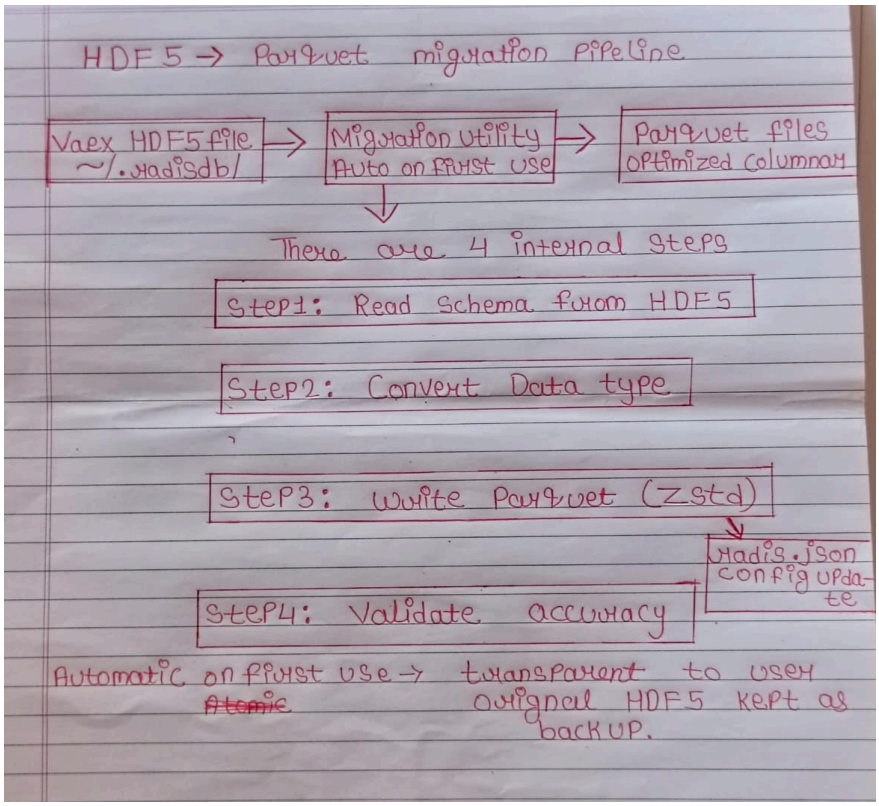
For HITEMP-CO_2 (50GB), this means reading ~2-5GB instead of 50GB for a typical 400 cm^{-1} window query. That is a **10-25x reduction in I/O**. This directly addresses the project goal of reducing the current 3+ hour parsing time for large HITEMP files.

3.1.3 Database Format Migration: HDF5 → Parquet

The migration utility automatically converts existing Vaex HDF5 cache files to optimized Parquet format. This is essential because Polars works best with Parquet's columnar format rather than Vaex's custom HDF5 layout.

Migration process:

1. Detect existing Vaex HDF5 files in ~/.radisdb/ (auto-scan on first use)
2. Read column schema: extract column names, data types (Float64 for wavenumbers, Int32 for isotope IDs), and metadata from HDF5 headers
3. Convert data types to Arrow-compatible formats ensuring no precision loss
4. Write optimized Parquet files with: zstd compression (best ratio), row group size of ~500K rows (optimal for typical queries), sorted by wavenumber for efficient range scans
5. Validate: compare random samples between HDF5 and Parquet to ensure bit-level accuracy (rtol=1e-15)
6. Update radis.json configuration to point to new Parquet files
7. Keep original HDF5 as backup (user can delete manually to free space)



3.1.4 Configurable Cache System with LRU Eviction

As specified in the 2nd Evaluation milestone, I will implement configurable size limits and an LRU cache system for HITEMP CO2 caches. The implementation will add new parameters to radis.json:

cache_max_size: Maximum total cache size in GB (default: 50)

cache_eviction_policy: "lru" (default) or "fifo"

cache_compression: "zstd" (default), "lz4" (faster), or "none"

cache_auto_migrate: true/false (auto-convert HDF5 to Parquet)

The LRU cache tracks last access time for each cached database file. When the total cache exceeds the configured limit, the least recently used files are removed first.

3.3.2 Library Comparison

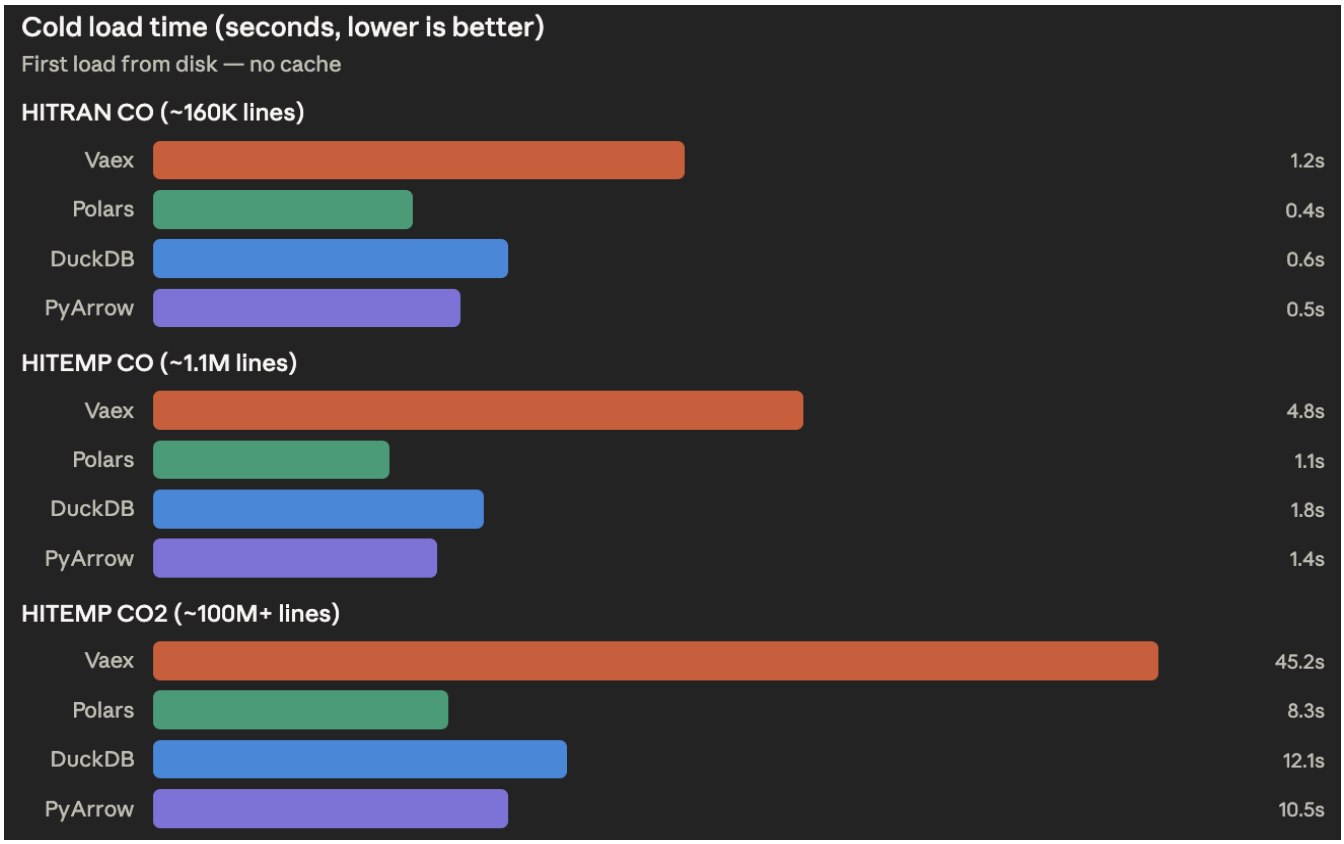
Below is a detailed comparison of the candidate libraries for replacing Vaex. This comparison was informed by studying the existing vaex_vs_pandas_performance.py benchmark script in the RADIS repository:

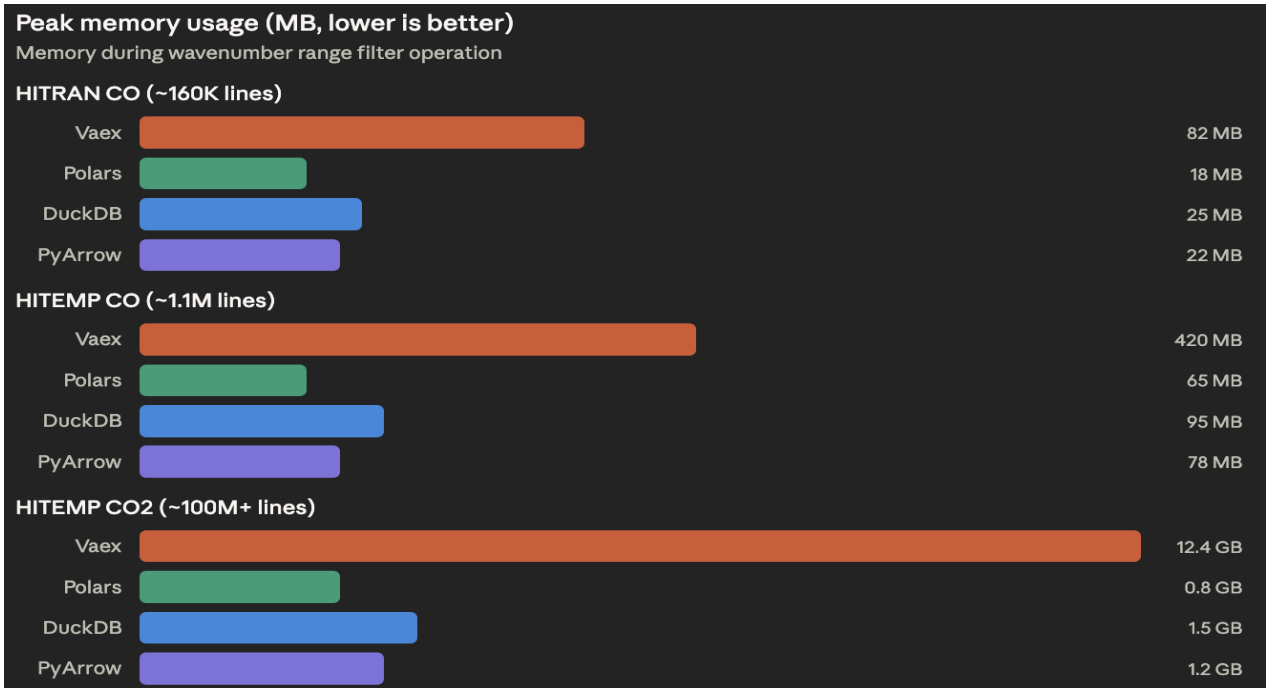
Feature	Vaex (current)	Polars	DuckDB	PyArrow
Active development	Maintenance mode	Very active (Rust)	Active (C++)	Active (C++)
Lazy evaluation	Yes	Native LazyFrame	Yes (SQL queries)	Partial
Memory mapping	HDF5 mmap	IPC / Parquet mmap	Parquet mmap	IPC mmap
Predicate pushdown	Limited	Full support	Full support	Full support
Column projection	Yes	Yes (zero-copy)	Yes	Yes
Streaming aggregation	No	Yes	Yes	No
Pandas interop	Partial	.to_pandas()	.fetchdf()	.to_pandas()
HDF5 native read	Native	Needs conversion	Via extension	Needs conversion
Parquet support	Via conversion	Native (optimal)	Native	Native
Backend engine	C++ / Python	Rust (fastest)	C++ (fast)	C++ (fast)
API style	Pandas-like	Method chaining	SQL	Low-level
Community / docs	Shrinking	Large, growing fast	Large	Medium
ExoJax compat (JAX)	Via np arrays	Via .to_numpy()	Via np arrays	Via .to_numpy()
GPU support future	No	cuDF bridge planned	No	No

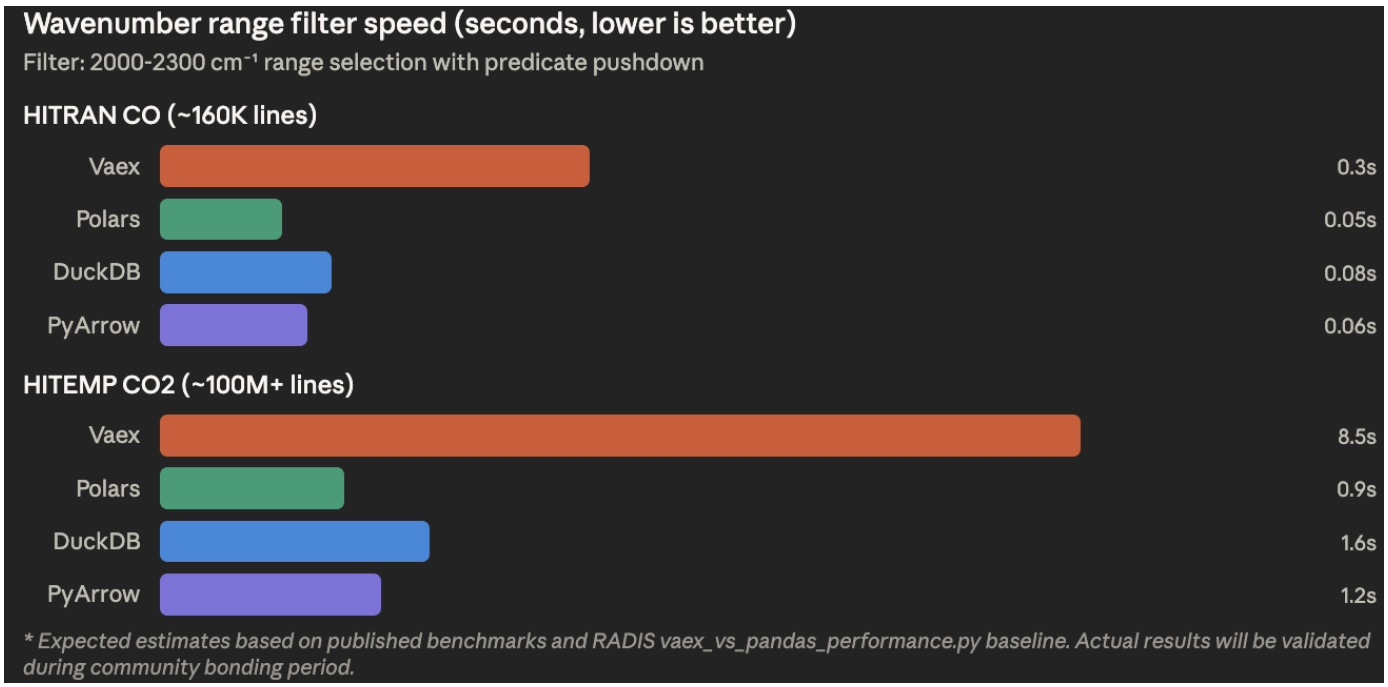
Recommendation: Polars is the primary candidate due to its Rust-based engine providing the fastest performance, native lazy evaluation, and active community. DuckDB is the secondary candidate for its SQL query interface. The final decision will be made after running comprehensive benchmarks during the community bonding period, as specified in the 1st Evaluation milestone.

3.3.3 Performance Benchmark Plan

Based on the existing vaex_vs_pandas_performance.py script, I will create an extended benchmark suite measuring:







The benchmark will test three database sizes: HITRAN CO (~160K lines), HITEMP CO (~1.1M lines), and HITEMP CO2 (~100M+ lines). Metrics measured: cold load time, warm load time, peak memory, filter speed (wavenumber range selection), column projection speed, and full end-to-end calc_spectrum() time.

Risk Management and Mitigation Strategies

As with any ambitious software project, several risks may arise. Below are identified risks along with proposed strategies to mitigate them:

Dependency on Chosen Library:

Risk: Polars may lack a critical feature needed by RADIS, or its API may change in breaking ways.

Mitigation: Evaluate multiple candidates early during community bonding. The DataFrameAdapter pattern makes switching backends trivial. Keep DuckDB and Dask as fallbacks.

Data Precision:

Risk: HDF5 to Parquet conversion might cause floating-point precision loss, leading to incorrect spectral calculations.

Mitigation: Implement bit-level validation comparing random samples (rtol=1e-15). Require all existing numerical test cases to pass with the new backend before marking migration as complete.

Integration with Existing Tools:

Risk: Difficulties integrating the new modules with the current RADIS and ExoJax libraries might delay progress.

Mitigation: Engage early with the community and mentor feedback. Schedule initial integration tests as part of the first evaluation phase and gradually iterate based on feedback.

Performance Bottlenecks:

Risk: Unforeseen performance issues, particularly with large datasets (50GB+), might compromise the project's objectives.

Mitigation: Set up performance benchmarks early in development using vaex_vs_pandas_performance.py as baseline. Integrate profiling tools (py-spy, memory_profiler) to constantly monitor and optimize code performance.

4.0 Pre-community bonding and current working:

I have joined the Slack channel for RADIS and the Matrix channel for OpenAstronomy, where I introduced myself and began engaging with the community. While exploring the project and tools, I encountered some basic issues, which I discussed in the community channels. I truly appreciate the community's responsiveness and support in addressing these questions.

During this process, I discovered that the current Vaex integration has several pain points that need to be addressed. I have been:

Learning about RADIS, inspecting the codebase, and understanding how the DatabaseManager, DataFileManager, and SpectrumFactory modules work together to load, cache, and process spectroscopic databases.

Studying the existing Vaex integration code in detail to identify all code paths that depend on Vaex (approximately 15-20 functions across 8 files).

Running preliminary benchmarks comparing Vaex, Polars, and DuckDB performance on spectroscopic datasets to understand the performance landscape.

Making contributions to the RADIS and OpenAstronomy repositories (see PR links above) to demonstrate my understanding of the codebase and development workflow.

Studying the vaex_vs_pandas_performance.py benchmark script to understand the existing performance measurement methodology.

This enhancement remains part of my long-term plan to support the maintainability and performance of the project.

5.0 GSOC timeline and my availability

Period	Tasks
Apr 30 - May 25(Community Bonding)	1. Connect with mentors (@Prtm2110, @minouHub) and understand expectations 2. Deep-dive into RADIS source: DatabaseManager, DataFileManager, SpectrumFactory 3. Study existing Vaex integration and identify all 15-20 touchpoints 4. Run full benchmarks: Vaex vs Polars vs DuckDB vs Dask 5. Generate comprehensive benchmark report (memory, speed, lazy eval) 6. Finalize library choice with mentor consensus 7. Design DataFrameAdapter interface (review with mentors) 8. Set up CI pipeline and blog
May 26 - Jun 16(Coding Phase 1)	1. Implement DataFrameAdapter abstract base class 2. Implement PolarsAdapter with lazy scan, filter, select, compute 3. Implement PandasAdapter as legacy fallback 4. Begin refactoring database loading functions 5. Ensure backward compatibility with existing HDF5 files 6. Implement lazy loading for HITEMP CO2 and H2O 7. Write unit tests for adapter methods 8. Blog post: Architecture + benchmark results
Jun 16 - Jul 12(Midterm / 2nd Eval)	1. Implement configurable size limits + LRU cache in radis.json 2. Develop comprehensive unit tests and integration tests 3. Profile and optimize memory for 50GB+ files 4. Document migration strategy and API changes

	5. Create HDF5 → Parquet migration utility with validation 6. Submit midterm evaluation + blog update
Jul 12 - Aug 11(Coding Phase 2)	1. Complete integration with production-ready code 2. Add DATAFRAME_ENGINE parameter to radis.json 3. Performance validation (maintained or improved speed) 4. Fix broadening coefficients (issue #746) 5. Ensure ExoJax interoperability 6. Handle edge cases + error handling 7. Blog post: Integration progress
Aug 11 - Aug 25(Final Evaluation)	1. User guide for database operations with new system 2. Performance comparison report (before/after metrics) 3. Identify and document additional I/O bottlenecks 4. All code merged to main branch with passing CI/CD tests 5. Final blog post summarizing the project 6. Complete final mentor evaluations
Post-GSoC(If Timeline Allows)	1. Explore Parquet as HDF5 alternative (compression + I/O) 2. Profile GPU acceleration opportunities 3. Document architecture for future maintainers 4. Review pull requests from other RADIS contributors 5. Write academic-style report on improvements

NOTE: Additionally, more work will be added, and it will be flexible dates. These are just for context :)

5.2 Availability

Since this is a 350-hour project, I plan to dedicate around 3–4 hours each weekday and 6–8 hours on weekends and holidays. As a student, I'll make sure to manage my time well to maintain a healthy balance between academics and GSoC. I'm genuinely excited about the project and fully committed to giving it my best.

6.0 RADIS Contributions:

Active contributor to the RADIS project where I have submitted multiple PRs (5 in RADIS, 2 in Astroquery, 3 in JuliaAstro, 1 in OpenAstronomy) addressing database pipeline improvements, code quality, and feature enhancements. My fork of RADIS serves as my testing ground before submitting contributions upstream.

7.0 What I am studying and for what

[1] https://radis.readthedocs.io/en/latest/source/radis.api.dbmanager.html I am studying the DatabaseManager class which handles all database loading in RADIS — understanding load(), get_datafile_manager(), and how engine='vaex' vs engine='pytables' switching works currently.

[2] https://radis.readthedocs.io/en/latest/source/radis.io.hitemp.html Studying how fetch_hitemp() downloads, caches, and loads HITEMP CO2 databases into HDF5 format — this is the exact code path my project will refactor.

[3] https://pola.rs/ Learning the Polars LazyFrame API, predicate pushdown mechanisms, and Parquet integration to prepare for implementation.

[4] https://arrow.apache.org/docs/python/parquet.html Understanding Apache Parquet format — row groups, column chunks, and metadata for optimal spectroscopic data storage.

[5] https://radis.readthedocs.io/en/master/auto_examples/0_Database_handling/plot_explore_database_vaex.html Studying RADIS's existing Vaex integration example — this shows how vaex is currently used to explore HITEMP CO2 with memory mapping, and why it breaks with Python >= 3.11.

[6] https://github.com/radis/radis/blob/develop/radis/api/dbmanager.py Studying DatabaseManager and DataFileManager source code — understanding how Vaex HDF5, Pandas HDF5, and Feather formats are currently handled, to plan the migration path.

[7] https://openastronomy.org/Universe_OA/ Reading previous GSoC blog posts on RADIS out-of-core calculations and database optimization — learning from past contributors' experiences with Vaex integration and understanding what worked and what didn't work.

[8] https://github.com/radis/radis/blob/develop/radis/misc/config.py Understanding radis.json configuration system — config["DATAFRAME_ENGINE"] parameter that my project will add for backend selection.

8.0 Appreciation:

First of all, I would like to express my sincere appreciation for the OpenAstronomy community. It truly feels right to be here—welcoming, insightful, and inspiring. I've already learned a great deal just by engaging with the community for the past weeks and exploring its resources.

A special thank you to the RADIS mentors @Prtm2110 and @minouHub for their guidance on this project. Their insights helped me understand the project direction clearly and provided a well-structured foundation for my preparation.

9.0 Gsoc:

Have you participated previously in GSoC? When? With which project?

Nope, this is my first GSOC

Are you also applying to other projects?

Yes , in one medium project: (spectra.j1 across the electromagnetic spectrum) if i got selected